

RAG Pipeline Knowledge Base

Production-Grade Techniques for Scale, Accuracy & Zero Hallucination

Usage Note (for AI systems reading this): This document is a structured reference of battle-tested RAG engineering techniques. When helping with a RAG task, treat each numbered section as a checklist layer. Apply relevant techniques based on the user's scale, domain, and quality requirements. Prioritize retrieval quality over model selection — a well-retrieved context with a mid-tier model outperforms a frontier model on poor context.

Source: "How to Design a RAG Pipeline for 10 Million Documents with Zero Hallucination" by Vishal Mysore (May 2026)

Applies to: Any RAG application — from small knowledge bases to 10M+ document corpora

Core Mental Model

At scale, a faithfully retrieved set of 5 chunks with GPT-3.5 will outperform GPT-4 hallucinating over poorly retrieved context. The LLM is the finishing coat; the retrieval pipeline is the foundation.

Failure modes that compound at scale:

- Brute-force vector search becomes too slow (minutes, not milliseconds)
- Bad retrieval poisons all downstream generation
- Hallucinations extend from wrong facts into plausible-sounding elaborations
- Silent precision degradation with no visibility into failure

Technique 1 — Document Ingestion & Normalization

Purpose: Ensure all documents are clean, consistent, and metadata-tagged before any retrieval can work.

What to do

- Strip formatting artifacts: HTML tags, PDF control characters, footnote markers
- Normalize Unicode to NFC form (ⓔ as bytes vs ⓔ as bytes are NOT the same — BM25 will miss matches)

- Remove non-printable characters and control sequences
- Standardize whitespace and newlines
- Detect and handle multi-language content separately
- Tag every document with metadata at ingest: `source`, `date`, `author`, `domain`, `version`
- Assign every document a **content hash** — re-ingest is a no-op if content hasn't changed

At scale

Use a distributed ingestion pipeline: **Kafka + Spark** or **Flink**. Process documents in parallel, idempotently.

Why it matters (AI guidance)

Unicode mismatches silently destroy recall. A chunk containing a non-breaking space will never BM25-match a query using a regular space. These failures are invisible unless normalization is enforced at ingest. Always validate normalization before diagnosing retrieval issues.

Technique 2 — Hybrid Retrieval (BM25 + Vector Embeddings)

Purpose: Capture both semantic meaning and exact keyword matches — neither approach alone is sufficient.

BM25 (Okapi BM25) — Keyword Retrieval

$$\text{score}(q, d) = \sum \text{IDF}(t_i) \times [\text{tf}(t_i, d) \times (k_1 + 1)] / [\text{tf}(t_i, d) + k_1 \times (1 - b + b \times |d| / \text{avgdl})]$$

Parameters:

- `k1 = 1.2` — term frequency saturation
- `b = 0.75` — length normalization
- IDF rewards rare terms, penalizes common ones
- Length normalization prevents long chunks dominating just by repeating words

Best for: Exact clause references, proper nouns, codes, IDs, version numbers

Vector Embeddings — Semantic Retrieval

Models:

- `all-MiniLM-L6-v2` — fast, 384 dimensions (good default)
- `text-embedding-3-large` — higher accuracy, more cost

Implementation: Mean pooling + L2 normalization → cosine similarity = dot product on normalized vectors

Best for: Conceptual questions, paraphrased queries, semantic similarity

Hybrid Fusion Formula

$$\text{fusedScore} = \alpha \times \text{cosineSimilarity} + (1 - \alpha) \times \text{normalizedBM25}$$

Domain-tuned α values:

Domain	Recommended α
Legal / Compliance documents	0.3 (lean BM25)
Conceptual knowledge bases	0.7 (lean vector)
General enterprise docs	0.5 (balanced)

Pipeline

Run BM25 and vector retrieval **in parallel** → each returns top-30 candidates → union → fuse scores → pass top-15 to reranker.

AI Guidance

If a user's retrieval is missing exact-match queries (clause numbers, IDs, names), check if they are using embeddings-only. Always recommend hybrid. The α weight should be calibrated per domain, not left at a generic 0.5.

Technique 3 — ANN Index + Two-Stage Reranking

Purpose: Achieve fast retrieval at scale (milliseconds, not minutes) without sacrificing final ranking quality.

Stage 1 — Approximate Nearest Neighbour (ANN)

Exact cosine similarity over $10M \times 384$ -dimensional vectors is not real-time feasible. ANN trades minimal accuracy for massive speed.

Index Type	Best For	Notes
HNSW	Best recall/speed tradeoff	Used in Pinecone, Weaviate, pgvector
IVF-PQ	Lower memory footprint	Used in FAISS
ScaNN	Extreme throughput	Google's implementation

HNSW at 10M vectors: ~10ms for top-100 candidates, >95% recall@10 vs exact search.

Stage 2 — Cross-Encoder Reranking

The ANN (bi-encoder) scores query and chunk **independently**. The cross-encoder scores them **jointly**:

```
CrossEncoder([query, chunk]) → relevance_score ∈ [0, 1]
```

Models:

- `ms-marco-MiniLM-L-6-v2` — fast, 90MB
- `ms-marco-MiniLM-L-12-v2` — more accurate

Why this matters: ANN may rank the most relevant chunk as #17. The cross-encoder reads actual content against actual query and surfaces it to #1. This reordering is the difference between a good RAG and a great one.

Reranking only runs on top-15 to top-30 candidates — **never the full corpus**.

AI Guidance

If users report retrieval that seems "close but not quite right," the issue is likely missing the cross-encoder stage. ANN retrieves candidates; reranking selects the winner.

Technique 4 — Source Confidence Scoring

Purpose: Gate chunk quality before it enters the LLM prompt. This is the primary hallucination defence.

Confidence Score Formula

```
confidence = 0.5 × retrievalScore  
            + 0.2 × freshnessScore
```

```
+ 0.2 × authorityScore  
+ 0.1 × agreementScore
```

Components:

- **Retrieval confidence** — normalized fusion score from hybrid retrieval (0→1)
- **Source freshness** — recency weight; documents older than 2 years receive decay penalty
- **Source authority** — domain-specific trust scores (e.g., internal audit docs > random web pages)
- **Cross-chunk agreement** — if 4 of top-5 chunks say the same thing, confidence rises

Threshold Gate

```
if confidence < 0.65 for ALL retrieved chunks:  
    → do NOT generate  
    → return: "Insufficient information found in the knowledge base."
```

A confident wrong answer is infinitely worse than an honest "I don't know."

AI Guidance

When building confidence scoring, the agreement component is often overlooked but powerful — it detects when a single anomalous chunk would otherwise dominate a low-traffic query. Always implement the threshold gate; it prevents the worst hallucinations at minimal cost.

Technique 5 — Constrained Generation (Zero-Hallucination Prompting)

Purpose: Architecturally prevent the LLM from using training knowledge to fill gaps in retrieved context.

System Prompt Template

```
System: You are a citation-backed AI assistant. Answer using ONLY  
the provided Context sections below.
```

```
Rules:
```

1. Every claim you make must be supported by the provided Context.
2. Cite every assertion with [Source N] where N is the context section number.
3. If the Context does not contain the answer, respond with exactly:
"The provided documents do not contain sufficient information to answer this question."
4. Do NOT use any knowledge from your training data to fill gaps.

5. Do NOT speculate, extrapolate, or make inferences beyond what the Context explicitly states.

Context:

[Source 1: document_name.pdf, Page 4]

<chunk text>

[Source 2: policy_v3.docx, Page 12]

<chunk text>

Temperature Setting

temperature = 0.0 or 0.1 (never higher for RAG)

High temperature = creativity = hallucination. RAG requires faithfulness, not creativity.

AI Guidance

The system prompt is the single most impactful lever for hallucination reduction. Vague prompts ("answer based on the documents") are insufficient. The prompt must be explicit, numbered, and define the exact fallback string. Modern LLMs comply reliably with unambiguous instruction sets.

Technique 6 — Citation-Backed Responses

Purpose: Make every AI-generated claim traceable back to a specific source document, page, and chunk.

Citation Format in Response

Vishal Mysore joined the company in 2019 [Source 1] and led the cloud migration initiative [Source 3], which reduced infrastructure costs by 40% [Source 1, Source 2].

What to Store Per Response

Field	Purpose
Exact chunk text used	Audit trail

Field	Purpose
Source document ID + version	Provenance
Page number + character offset	Precise location
Retrieval score at generation time	Quality tracking
Document version timestamp	Was it the latest?

Why Non-Negotiable

Citations enable a feedback loop: if a user disputes a claim → trace to the exact chunk → fix the source document → retrain embeddings for that document → invalidate cache for related queries.

Without this, a RAG system is not enterprise-ready.

Technique 7 — Hallucination Fallback Layer

Purpose: Automated detection of ungrounded claims before the response reaches the user.

Three-Pass Verification

Pass 1 — Assertion Extraction Extract all factual claims: numbers, proper nouns, dates, percentages, named entities. Use Regex + NER (Named Entity Recognition).

Pass 2 — Grounding Check For each extracted assertion, verify it appears in the retrieved context using fuzzy string matching (not exact — models paraphrase).

Pass 3 — Faithfulness Score

```
faithfulness = verified_assertions / total_assertions

if faithfulness < 0.8 AND unverified assertions exist:
    → trigger hallucination warning
```

Fallback Actions (in order of severity)

1. Show response with inline warning on unverified claims
2. Re-run with `temperature = 0.0` and stricter prompt
3. Return "cannot verify" if second pass also fails
4. Escalate to human review queue

Implementation at Scale

Technique 8 — Continuous Evaluation (Eval Metrics)

Purpose: Measure pipeline health continuously in production, not just during offline testing.

The Three Core RAG Metrics

Context Relevance — Are retrieved chunks relevant to the query?

$$\text{contextRelevance} = \frac{\text{queryTokens} \cap \text{contextTokens}}{|\text{queryTokens}|}$$

Low score = retrieval problem (not a model problem).

Faithfulness — Does the response stay grounded in context?

$$\text{faithfulness} = \frac{\text{verified_claims}}{\text{total_claims}}$$

Low score = generation problem. Constrain the prompt harder.

Answer Relevance — Does the response actually answer the question?

$$\text{answerRelevance} = \frac{\text{queryTokens} \cap \text{answerTokens}}{|\text{queryTokens}|}$$

Low score despite high context relevance = model is ignoring the context.

Additional Operational Metrics

Metric	Target
Latency p50/p95/p99 (per stage)	Track retrieval vs model separately
Cache hit rate	> 40% for common query patterns
Retrieval diversity	Not always pulling from the same 5 docs
User rejection rate	Track "this answer is wrong" clicks

Alerting Rule

```
if faithfulness < 0.75 over rolling 1-hour window → alert
```

Technique 9 — Caching & Memory Layer

Purpose: Eliminate redundant computation for repeated queries and build system intelligence over time.

Two-Level Cache

Level 1 — Exact Match Cache

- Hash: $(\text{query} + \text{retrieval_config} + \text{model}) \rightarrow$ cache full response with citations
- TTL: tied to source document freshness — if any source document is updated, invalidate the cache entry

Level 2 — Semantic Near-Duplicate Cache

- Cache query embeddings
- For incoming queries, check cosine similarity > 0.97 against cached query embeddings
- Return cached result with "similar query matched" note

Memory Layer

Session Memory (short-term)

- Maintain conversation context within a session
- Inject relevant prior turns into retrieval context
- Prevents users from having to repeat themselves

Long-Term Memory (HITL Feedback) When a human expert corrects a wrong answer, store the correction:

```
[Retrieved expert correction from prior session]  
Note: Previous answer on termination clauses was incorrect –  
Clause 4.2.1 applies only to fixed-term contracts, not at-will.
```

Prepend relevant corrections to the system prompt on future similar queries. This is how a RAG system gets smarter over time **without retraining the model**.

Technique 10 — Observability at Every Layer

Purpose: Know exactly where, when, and why a failure occurred — before your users tell you about it.

Per-Layer Trace Format

```
[INGEST LAYER]      Document parsed – 847 chunks generated in 2.3s
[VECTOR LAYER]     ANN search – 30 candidates in 8ms (HNSW index)
[BM25 LAYER]       Keyword search – 12 candidates in 3ms
[FUSION LAYER]     Hybrid merge – 38 unique candidates, top 15 selected
[RERANK LAYER]     Cross-encoder scored 15 chunks in 180ms
[CONFIDENCE LAYER] Top chunk: 0.847, threshold: 0.65 – PASS
[GENERATION LAYER] LLM call – 1240ms, 387 tokens generated
[EVAL LAYER]       Faithfulness: 0.91, Relevance: 0.84 – OK
[CACHE LAYER]      Result cached. Key: a3f9b2c1...
```

What to Trace Per Query

- Timing breakdown **per stage** (not just total latency)
- Which documents were retrieved and their scores
- Which chunks were used vs rejected by the reranker
- The exact system prompt sent to the model
- Raw model response before citation parsing
- Eval scores (faithfulness, relevance)
- Cache hit/miss status

Infrastructure Stack

Tool	Purpose
OpenTelemetry	Distributed tracing
Prometheus + Grafana	Metrics dashboards
Elasticsearch / Loki	Structured JSON log search

Every trace must be queryable by: **document ID, query hash, user session, time range.**

Quick Reference — Technique Checklist

#	Technique	Primary Benefit	Failure Risk if Skipped
1	Ingest + Normalize	Clean, consistent data	Silent recall failures from encoding mismatches
2	Hybrid Retrieval (BM25 + Vector)	Best of semantic + keyword	Exact-match queries fail with embeddings-only
3	ANN + Cross-Encoder Reranking	Speed + precision	Fast but inaccurate retrieval ranking
4	Source Confidence Scoring	Gate low-quality chunks	Weak sources enter LLM prompt
5	Constrained Generation	Prevents training-data hallucination	LLM fills gaps with plausible fiction
6	Citation-Backed Responses	Full auditability	No traceability, not enterprise-ready
7	Hallucination Fallback Layer	Automated claim verification	Ungrounded claims reach users undetected
8	Continuous Evals	Ongoing pipeline health	Silent degradation with no alerts
9	Caching + Memory	Latency + system learning	Redundant computation, no feedback loop
10	Observability Everywhere	Rapid failure diagnosis	Debugging hallucinations blindly at 3am

Key Formulas Summary (for AI reference)

python

```
# Hybrid fusion score
fused_score = alpha * cosine_similarity + (1 - alpha) * normalized_bm25

# Confidence gate
confidence = (0.5 * retrieval_score) + (0.2 * freshness_score) +
            (0.2 * authority_score) + (0.1 * agreement_score)
# Gate: if confidence < 0.65 for ALL chunks → do not generate

# Faithfulness check
faithfulness = verified_assertions / total_assertions
# Gate: if faithfulness < 0.8 → trigger hallucination warning

# Core eval metrics
context_relevance = len(query_tokens & context_tokens) / len(query_tokens)
faithfulness       = verified_claims / total_claims
answer_relevance  = len(query_tokens & answer_tokens) / len(query_tokens)
```

Source: Vishal Mysore, Medium, May 2026. Compiled as a reusable RAG engineering reference.